

Entrada-Salida

Un sistema de computación generalmente incluye varios dispositivos de entrada-salida (E/S) y de diferentes tipos. Una clasificación se basa en la dirección de transmisión de los datos. Si sólo es posible leer datos del dispositivo (de entrada), escribir (de salida) o ambos (entrada-salida).

Otra clasificación se basa en los modos de transferencia.

1. *Dispositivos de caracteres (bytes)*
2. *Dispositivos de bloques*

En el primer caso el dispositivo transfiere de a bytes, como por ejemplo un teclado, mouse o un dispositivo de comunicaciones tipo UART.

Un dispositivo de bloques, como por ejemplo un disco, transfiere de a bloques comúnmente de tamaño fijo (por ejemplo de 512 bytes).

Hay algunos dispositivos que no encajan bien en esta clasificación, como por ejemplo, algunos dispositivos de comunicaciones de red, como Ethernet. Estos dispositivos transmiten en forma serial bloques de tamaño variable denominados *paquetes*. Estos *packets* comúnmente tienen un tamaño mínimo y uno máximo.

Otro dispositivo que comúnmente se maneja de manera especial es el *timer*, el cual se programa para que genere interrupciones periódicamente para que el kernel pueda tomar el control.

Interconexión de dispositivos

Una arquitectura de hardware puede interconectar sus dispositivos a un bus de E/S o éstos pueden aparecer conectados directamente en algún área física del espacio de direcciones de memoria (*memory mapped input-output o mmio*). Esta interconexión al espacio de direcciones se puede hacer *hard-wired* aunque las arquitecturas modernas usan un *memory controller chip* programable que permite configurar y rutear requerimientos de lectura/escritura a diferentes *chips* (ROM, DRAM y dispositivos de I/O) según el valor en el bus de direcciones

A modo de ejemplo, la arquitectura x86 (Intel) es híbrida. La CPU contiene un bus de E/S separado del bus de memoria. Algunos dispositivos se conectan al bus de E/S (que tiene un espacio de direcciones de 16 bits) mientras que otros se *mapean en memoria (mmio)*. La cpu ofrece instrucciones especiales, como `in` y `out` para transferir datos por el bus de E/S.

En una PC basada en x86 el *frame buffer* del controlador de video (CGA, VGA, etc) se *mapea en memoria*. Al escribir datos en ese espacio de direcciones reservado para ese dispositivo para producir una salida en el display o pantalla.

El *board* risc-v usada en el taller, no dispone de un *video controller*, pero soporta la conexión de una terminal (*tty*) por su puerto serie (UART). La terminal transmite los códigos de las teclas pulsadas en el teclado y muestra en pantalla las líneas de caracteres recibidos. La interface de programación del UART permite transmitir y recibir bytes. Se basa en un protocolo de comunicación muy simple y transmite físicamente cada bit de un byte en forma serial. Para más detalles ver [UART](#).

Varias arquitecturas modernas, como RISC-V, tienden a usar el mecanismo de *mmio* ya que se puede aprovechar el inmenso espacio de direcciones lógicas que proveen. Esto además simplifica el diseño del hardware y elimina muchos componentes requeridos para implementar un bus adicional.

En la [figura 4](#) se muestra el mapping de algunos dispositivos en la arquitectura risc-v.

A modo de ejemplo, el controlador de video [VGA](#) en la IBM PC, expone el *text buffer* de video en modo texto (en color) en la dirección física `0xB800` y tiene un tamaño dependiendo del modo usado (comúnmente de 80 columnas por 25 filas). Cada caracter en pantalla se representa en 16 bits de la forma (*ascii,attributes*), con *ascii* siendo el código [ascii](#) del caracter y los *attributes* describen el color de fondo, frente y el parpadeo. En [DOS](#) o en las primeras versiones de *MS-Windows* las aplicaciones podían leer o escribir directamente en el buffer (sin protección).

El hardware gráfico moderno soporta modo gráfico (matriz de *pixels*). El *modo texto* (usado en consolas) se implementa *dibujando* los caracteres gráficamente. La *forma* de cada caracter se describe en un *font*. Un *font* describe los caracteres en forma de matriz de pixels (*raw*) o en forma vectorial (*paths* de trazado). Esta última representación permite un mejor *escalado*.

Buses de interconexión

La mayoría de las arquitecturas soportan la conexión de buses de E/S. Un bus a su vez es un dispositivo en sí mismo con su propio controlador programable. Un bus de E/S permite que se conecten a él varios dispositivos y a su vez otros buses secundarios como se muestra en el siguiente diagrama.

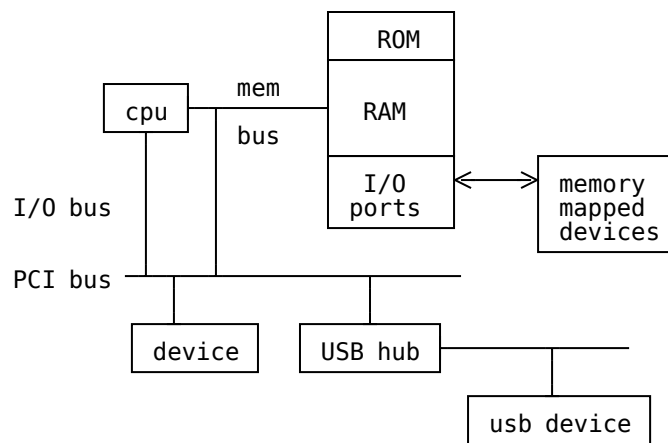


Figura 1: Buses de interconexión (I/O dedicada y mmio).

Un bus estándar muy usado es el *Peripheral Component Interact (PCI)* al cual comúnmente se usa como bus principal. Este bus permite conectar dispositivos como interfaces de red, controladoras de disco, co-procesadores gráficos, dispositivos de audio, etc. Permite a su vez que se conecten controladores de otros buses como *Universal Serial Bus (USB)* al cual soporta un árbol de *USB hubs*. Un *USB hub* contiene un conjunto de *USB ports* en los cuales se conectan los dispositivos físicos.

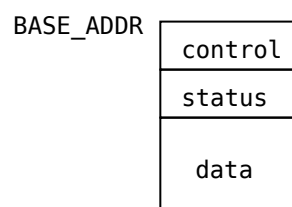
Los estándares de buses como PCI o USB describen sus características físicas como conectores, niveles eléctricos, etc y además su interfaz de control (registros de control, estado y datos y detalles de su programación).

Cada dispositivo compatible, en el inicio (o en el momento de su conexión) se *anuncia* en el bus, con información como el tipo de dispositivo, fabricante, registros de E/S, líneas de interrupción a usar, etc. El controlador del bus registra esta información en una tabla en memoria (RAM o registros propios) permitiendo que el SO *descubra* o *auto-detecte* los dispositivos existentes.

Algunos buses permiten la conexión en *caliente*, es decir mientras el hardware está encendido. En estos casos el bus *notifica* (genera una *interrupción*) al kernel indicando que un nuevo dispositivo se conectó permitiendo que el OS active el *device driver* correspondiente. Esto se conoce como dispositivos *plug-and-play* (enchufar y usar).

Control de dispositivos

Un dispositivo expone un conjunto de *puertos* (o registros) de control y estado. Esos puertos están contenidos en el *controlador del dispositivo* (*device controller*).



Algunos registros, denominados de control, se usan para enviar comandos al dispositivo. Otros son registros de datos, contienen los datos a ser transferidos o recibidos. Otros registros son de estado, indicando el estado del dispositivo como por ejemplo si un dato está disponible, si se produjo algún tipo de error o si el dispositivo está configurado para generar interrupciones o no.

Estos puertos aparecen en el espacio de direcciones del bus de E/S o de memoria, según su forma de interconexión.

La documentación técnica de cada dispositivo (o tipo si se basa en algún estándar) describe los detalles de cómo un SO debería interactuar y controlar el dispositivo. Es común que las direcciones de los puertos del controlador del dispositivo sea dependiente de la plataforma de hardware.

Device drivers

El módulo del kernel que controla un tipo o familia de dispositivos se denomina *device driver*. Un SO define la interface (funciones) que debe implementar el device driver. El kernel interactúa con un device driver mediante esta API.

Se sigue un diseño polimórfico al estilo orientado a objetos en el cual cada device driver expone una estructura de datos con punteros a sus funciones que implementan la API.

Comúnmente un *device driver* debe implementar las siguientes funciones:

- *Detección (discovery)* de los dispositivos que puede controlar.
- *Inicialización* de los dispositivos. Esto comúnmente involucra la configuración de las interrupciones que debe generar, informar al kernel las direcciones de E/S que usará (reserva), y otros detalles.
- *Operaciones de lectura y/o escritura*: Implementar el control y programación de los requerimientos de transferencias (*i/o requests*).
- *Manejar las interrupciones* generadas por el dispositivo. Comúnmente, cuando un dispositivo completa una operación, éste genera una interrupción.

Una forma alternativa de determinar si una operación se completó es *pooling*, es decir, verificar periódicamente un bit de un *registro de estado* del dispositivo que lo indique.

En el capítulo 5 del manual de xv6 se dan algunos detalles técnicos sobre los dispositivos soportados y la implementación de sus *device drivers*.

Diseño del subsistema de E/S

Dada la amplia variedad de dispositivos que soporta un SO es necesario un diseño que permita abstraer a los device drivers a las capas superiores del kernel.

Comúnmente el diseño del subsistema de E/S se basa en que cada device driver se *registra* en el sistema y reserva los recursos requeridos como espacios de direcciones a puertos/registros del controlador y qué línea (o número) de interrupción usará.

Así el subsistema de I/O mantiene una tabla de dispositivos. La estructura de datos que representa un dispositivo contiene las direcciones de puertos de I/O, líneas de IRQ a usar y otros. Además, cada dispositivo se asocia a un *device driver* representado por una estructura de datos con punteros a sus operaciones soportadas.

Interface de un device driver

Para facilitar la interoperabilidad entre los diferentes módulos del kernel con los device drivers. Cada device driver es un módulo que debe implementar una interface definida en el SO (comúnmente por el filesystem). Es común que se defina una interface para cada tipo de dispositivos. En los SO tipo UNIX hay básicamente tres clases de dispositivos.

1. Dispositivos de *caracteres*: La entrada-salida es de a bytes, como por ejemplo, dispositivos como el teclado, mouse, puerto serie.
2. Dispositivos de *bloques*: La lectura/escritura es por bloques, como por ejemplo en los discos (almacenamiento no volátil). Comúnmente el filesystem ofrece una caché para la E/S de bloques.
3. Dispositivos de *comunicaciones* o de *red*: Como interfaces de red tipo Ethernet, Wifi, Bluetooth y otros.

Más adelante se describe la interface de dispositivos de bloques en GNU-Linux.

Software independiente de dispositivos

El subsistema de entrada/salida además provee algunos servicios o políticas que son independientes de los dispositivos como planificar los requerimientos de E/S y manejo de buffers de E/S.

A modo de ejemplo, es común que se provea el servicio de *disciplina de líneas* (*line discipline*) entre *tty device drivers* y algunas llamadas al sistema como *read/write* sobre terminales de texto. Una disciplina de líneas típica realiza entrada/salida de texto por líneas (cadenas de caracteres).

Una disciplina de líneas de entrada (input) tiene en cuenta caracteres de control de entrada como **Ctrl-D** , **Enter** , **BS** (backspace) y otros. Caracteres como el generado por la tecla **Enter** indica fin de la línea, **Ctrl-D** indica fin de la entrada y **BS** borra el caracter ingresado previamente.

Comúnmente la implementación de esta disciplina se basa en la gestión de un buffer de caracteres de entrada organizado como una cola circular.

En la salida (output) se tienen en cuenta los efectos de los caracteres de control como `CR` (carriage return) y `LF` (linefeed), que producen bajada de cursor y desplazamiento de líneas (scroll) en pantallas o avance y retorno de carro.

En xv6, en el módulo `uart.c` se implementa el driver y la disciplina de líneas sobre una terminal (`tty`) conectada al puerto serie de tipo `UART`.

Buffers y cache

Para minimizar los accesos a dispositivos (como los discos) cada vez que se lee un bloque se cargan en bloques en memoria (comúnmente páginas) que se manejan como una caché. Esa caché generalmente tiene un límite de bloques.

Los bloques asignados de caché se reúsan y comúnmente se usa un algoritmo LRU para elegir una víctima para liberar en la caché.

Llamadas al sistema del tipo `flush(fd)` o `close(fd)` producen que se transfieran los bloques de cache modificados del archivo a disco.

La entrada-salida de dispositivos de caracteres comúnmente no utiliza buffers de caché.

Planificación de E/S

El SO debe gestionar los requerimientos de E/S realizados por los procesos (y del kernel). Comúnmente los requerimientos dependerán de cada tipo de dispositivo pero se pueden definir algoritmos de planificación que pueden ser comunes o particulares para diferentes tipos de dispositivos.

Algunos de estos algoritmos son similares a los de planificación de procesos.

- *FIFO*: Los requerimientos se procesan o envían al *device driver* en orden de llegada.
- *Prioridades*: Los requerimientos se ordenan por prioridades.

Para los dispositivos de bloques electromecánicos como los discos magnéticos, los requerimientos pueden ordenarse por la cercanía de la posición actual del brazo de lectura/escritura del dispositivo. Procesar los requerimientos más cercanos a la posición actual del brazo reduce significativamente los tiempos de acceso. Esto constituye un sistema de prioridades. Para evitar el problema de que los requerimientos de sectores alejados de la posición (*cilindro*) actual sean demorados se usa el *algoritmo del elevador*.

Algoritmo del elevador

Este algoritmo lleva una variable *direction* que representa la dirección del brazo de lectura/escritura. Los requerimientos se ordenan y se procesan en una dirección. Al llegar al cilindro del extremo se invierte la dirección. Luego comienzan a procesarse los demás requerimientos en base al orden de cercanía.

Transferencia directa a memoria (DMA)

Una forma de transferir datos desde/hacia la memoria RAM a los buffers internos de un dispositivo es por software, es decir mediante una operación o función que copia los datos de la forma `memcpy(dst, src, count)` .

Si un dispositivo soporta *direct memory access (DMA)* es posible evitar las copias por software (que puede insumir muchos ciclos de cpu) haciendo que el dispositivo realice la transferencia de manera autónoma. Esto permite *paralelizar* la ejecución de instrucciones de la cpu con transferencia de datos entre la memoria y el dispositivo.

Para esto, el *device driver* le debe indicar al dispositivo DMA la dirección física del buffer en memoria y su tamaño para luego indicarle que inicie la transferencia de datos. El dispositivo DMA competirá con la CPU por el acceso a la memoria usando una técnica de *robo de ciclos*.

Comúnmente, el dispositivo DMA genera una interrupción cuando la transferencia finalice. Actualmente, muchos dispositivos ya cuentan con un mecanismo DMA incluido en el mismo chip.

Integración con el filesystem

El sistema de E/S se relaciona con el sistema de archivos para brindar una abstracción: Los dispositivos son archivos especiales y los procesos pueden acceder a ellos (a través de los device drivers) por medio de la API de operaciones sobre archivos (`open()`, `read()`, `write()`, `...`).

En el capítulo [sistemas de archivos](#) se dan mas detalles sobre esta integración.

El sistema de archivos define una interfaz: Un conjunto de operaciones que cada device driver de ese tipo debe implementar.

```
struct file_operations {  
    struct module *owner;  
    loff_t (*llseek) (struct file *, loff_t, int);
```

```

    ssize_t (*read) (struct file *, char *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
    int (*readdir) (struct file *, void *, filldir_t);
    unsigned int (*poll) (struct file *, struct poll_table_struct *);
    int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, struct dentry *, int datasync);
    int (*fasync) (int, struct file *, int);
    int (*lock) (struct file *, int, struct file_lock *);
    ssize_t (*readv) (struct file *, const struct iovec *, unsigned
        loff_t *);
    ssize_t (*writev) (struct file *, const struct iovec *, unsigned
        loff_t *);
};

```

Listado 1: Ejemplo de estructura `file_operations` de Linux.

Los dispositivos se *presentan* en el sistema de archivos como archivos especiales. En los sistemas tipo UNIX, los archivos que representan dispositivos están creados en `/dev`. El tipo de cada archivo determina el tipo de dispositivo (`c` si es de caracteres o `b` si es de bloques).

Un proceso de usuario puede usar las llamadas al sistema `open`, `read`, `write`, `close`, ... para interactuar directamente con el device driver del kernel. Por ejemplo, el siguiente comando UNIX escribe en la consola o terminal de texto (*teletype* o *tty*).

sh

```

$ echo hello > /dev/tty
hello

```

En el caso que se lea desde un dispositivo, el proceso podría recibir los datos *crudos* del dispositivo, como se muestra en la siguiente demostración.

Demo 1: Leyendo del mouse.

En los sistemas tipo UNIX, un archivo especial (device) contiene sólo sus metadatos (no almacena datos en disco). Entre los metadatos más importantes son su tipo (`c` , por caracteres o `b` , por bloques), su *major number* que representa el tipo de dispositivo y le permite al kernel identificar el *device driver* correspondiente.

El *minor number* representa el número de dispositivo dentro de los de su tipo.

El *minor number* se pasa en cada operación (en un campo de la estructura `inode` en el listado 1) al device driver para que éste identifique el dispositivo con el que debe interactuar para realizar la operación.

Dispositivos virtuales

Es común que un SO implemente dispositivos implementados (o simulados) en software como por ejemplo:

- *random*: Generador de números aleatorios. De sólo lectura.
- *zero*: Retorna ceros. Sólo lectura.
- *null*: Ignora todo lo que recibe. De sólo escritura.

Otros dispositivos virtuales (por software) en realidad comprenden varios dispositivos físicos y usan los device driver correspondientes. Por ejemplo una *consola* o terminal (`tty`) consiste de un teclado y un display. El device driver de una `tty` comúnmente define e implementa su control y asociación con los dispositivos. Un driver de este tipo comúnmente soporta *consolas o terminales virtuales*. Así el OS permite a un usuario crear varias consolas o terminales como espacios de trabajo diferentes.

Otro ejemplo son los *redundant array of inexpensive disks (RAID)*. Un dispositivo RAID consiste en un conjunto de discos presentados al sistema como un único disco. Un RAID representa un dispositivo de almacenamiento tolerante a fallas y que permite accesos en paralelo.

Un RAID permite *distribuir* y *replicar* los bloques de datos de archivos en los discos. Un archivo con bloques de datos distribuidos en diferentes discos permite que se acceda en paralelo a ellos.

La replicación de bloques permite que ante una falla de un disco puedan accederse a alguna *copia* en otro disco, permitiendo al sistema continuar funcionando. Para más información sobre RAID y sus posibles configuraciones, ver [RAID](#).

Si bien actualmente existe hardware que soporta directamente RAIDs, en algunos OSes, como en GNU-Linux, es posible hacerlo por software usando discos estándares (ej: ATA).

Device drivers cargables dinámicamente

La mayoría de los SO modernos permiten la *carga* de *device drivers* dinámicamente. Esto significa que es posible compilar un device driver de un kernel como un *kernel module*.

En un diseño *microkernel* ésto es normal ya que cada *device driver* debería ser un componente (programa) independiente del resto del kernel y éstos se comunican por mensajes. Por ejemplo, en MS-Windows los drivers toman la forma de *dynamic link libraries* (DLLs).

En un sistema monolítico, esto es más complicado ya que estos módulos deberían *enlazarse dinámicamente* con el código del kernel para *aparecer* en su mismo espacio de memoria.

En GNU-Linux son *shared objects* compilados y enlazados como una *shared library* con extensión *.ko* (*kernel objects*) denominados *Linux Kernel Modules* (LKM). El comando `insmod` mapea el archivo objeto (driver) en memoria y ejecuta la llamada al sistema `init_module()`. Esta llamada al sistema realiza el enlazado dinámico del módulo en espacio del kernel e invoca a la función `init()` del módulo.

GNU-Linux contiene los comandos de usuario `lsmod`, `insmod` y `rmmod` para listar, cargar y remover módulos cargables dinámicamente.

Estándares

Actualmente se han definido varios estándares para diferentes tipos de dispositivos. Estos estándares especifican detalles técnicos de bajo nivel (niveles eléctricos, conectores, etc) y lógicos, como los puertos de control y estado que el controlador debe exponer y los detalles de su configuración y operación.

Que los dispositivos adhieran a estándares simplifica la escritura de los *device drivers* ya que deben basarse en la documentación del estándar.

Algunos estándares de dispositivos se listan a continuación:

- *Integrated Drive Electronics (IDE)*: Interface estándar para dispositivos de almacenamiento masivo (discos).
- *AT Attachment*: Evolución de IDE. Define dispositivos con transferencia en paralelo (*PATA*) y en serie (*SATA*). Para mayores detalle, ver [IDE-ATA](#)
- *Universal Asynchronous Receiver-Transmitter (UART)*: Dispositivos de comunicación serial. Para más detalles, ver [UART](#)
- *ISO/IEC JTC*: Estándares para equipos de oficina como impresoras y escáners.
- *VirtIO*: Interface de dispositivos virtuales. Comúnmente usados en SO corriendo en máquinas virtuales. Este estándar define diferentes tipos de dispositivos y describe sus interfaces lógicas (puertos y operación). No incluye detalles físicos ya que tiene como objetivo definir dispositivos virtuales (comúnmente emulados por software). El estándar actual es [virtio-1.2](#).

VirtualIO interface

Este estándar define una interface unificada para diferentes tipos de dispositivos. Los dispositivos aparecen mapeados en memoria o conectados a un bus PCI y exponen su tipo (block, network, input device, ...), estado, características soportadas, notificaciones, espacio de configuración y una o más *virtual queues*.

Una *virtual queue* permite gestionar buffers de entrada salida compartidos entre el *device driver* y el *device controller* o dispositivo.

Con la mayoría de los dispositivos como por ejemplo discos, comúnmente el device driver usa una *virtual queue* con los requerimientos y buffers de lectura y escritura.

Con los dispositivos de red comúnmente se usan dos *virtual queues*: Una para los paquetes recibidos y otra para los paquetes a enviar.

Para más detalles de la especificación, ver [virtio](#)